

Arquitetura de Integração (Singleton e Factory Method)

1. Objetivo da Atividade

A presente atividade visa avaliar a capacidade dos discentes em conceber e implementar uma solução arquitetural baseada em Padrões de Projeto Criacionais. O foco reside na resolução de problemas reais de concorrência, esgotamento de recursos e acoplamento em sistemas corporativos de alta disponibilidade, exigindo a aplicação justificada dos padrões **Singleton** e **Factory Method**.

2. Contextualização do Cenário Corporativo

O projeto envolve a modernização do núcleo de transações financeiras de um Gateway de Pagamentos Omnichannel (denominado *PagCore*), responsável por processar milhares de requisições por minuto para grandes plataformas de e-commerce.

Atualmente, o *PagCore* enfrenta dois gargalos arquiteturais severos que ameaçam a continuidade do negócio:

Problema A: Bloqueio Regulatório por Esgotamento de Conexões

Por exigência do Banco Central, toda transação processada deve ser registrada instantaneamente em uma API de *Compliance* e Prevenção à Lavagem de Dinheiro (Coaf). A arquitetura atual instancializa um novo objeto de conexão e abre um novo socket TCP/IP para cada transação. Como resultado, o sistema frequentemente ultrapassa o limite de conexões simultâneas permitido pela API governamental, ocasionando o bloqueio temporário do IP da empresa e a consequente paralisação de todas as vendas.

Problema B: Rigidez e Risco na Adição de Meios de Pagamento

O *engine* central de roteamento do sistema é composto por um bloco condicional maciço (switch/case com centenas de linhas). Toda vez que o setor comercial fecha parceria com um novo adquirente ou institui um novo meio de pagamento (ex: *Criptomoedas* ou *Buy Now, Pay Later*), os engenheiros são forçados a alterar a classe central de roteamento. Isso gera conflitos de integração, fere o Princípio do Aberto/Fechado (OCP) e frequentemente introduz falhas em meios de pagamento que já operavam de forma estável (como PIX e Cartão de Crédito).

3. O Desafio Arquitetural (Escopo da Entrega)

A equipe de engenharia deve projetar uma nova topologia para o módulo de processamento, garantindo que o sistema seja escalável, resiliente e imune aos problemas supracitados.

A solução **depende obrigatoriamente** da correta implementação estrutural dos seguintes

padrões para funcionar:

3.1. Requisito Estrutural 1: Gerenciamento Centralizado de Compliance (Singleton)

Deve-se projetar um `GerenciadorDeCompliance`. Este componente deve ser o único ponto de acesso à API do Banco Central em toda a infraestrutura do *PagCore*.

- **Restrição Técnica:** O sistema não pode, sob nenhuma hipótese de concorrência (*multithreading*), permitir a existência de mais de uma instância deste gerenciador em memória.
- **Justificativa Funcional:** Apenas uma instância garante que o controle de fluxo (uso de uma única fila de envio em *background*) seja respeitado, evitando múltiplos sockets e o bloqueio do IP da organização.

3.2. Requisito Estrutural 2: Roteamento Dinâmico de Transações (Factory Method)

A classe central do Gateway (o *Processador de Transações*) deve receber requisições de pagamento padronizadas (contendo o valor e o tipo de método escolhido) e processá-las sem conhecer a implementação técnica da integração bancária.

- **Restrição Técnica:** O *Processador de Transações* deve delegar a criação do objeto de pagamento específico para uma estrutura de Fábrica. Deve ser criada uma interface ou classe abstrata base (`ProcessadorPagamento`) e classes concretas (`ProcessadorPix`, `ProcessadorCartao`, `ProcessadorSwift`).
- **Justificativa Funcional:** Ao isolar a lógica de instanciação em um `Factory Method`, a adição futura de um "Processador de Criptomoedas" exigirá apenas a criação de uma nova classe concreta e o registro na fábrica, mantendo o *engine* central de roteamento estritamente fechado para modificações (OCP).

4. Instruções e Entregáveis

Os discentes não receberão código-fonte base, devendo construir a arquitetura a partir de um projeto em branco. A entrega final deve conter:

1. **Documentação Arquitetural (Diagrama UML):**
 - Um Diagrama de Classes detalhando as interfaces, as classes concretas, a Fábrica (Factory) e o componente de integração governamental (Singleton).
 - O diagrama deve evidenciar as relações de herança, implementação e dependência, além da visibilidade correta de atributos e construtores essenciais para o funcionamento dos padrões.
2. **Implementação do Código-Fonte:**
 - Estruturação dos pacotes refletindo a separação de responsabilidades.

- Implementação do Singleton com tratamento de concorrência (*Thread-safety*), justificando tecnicamente (via comentários em código) a abordagem utilizada (ex: *Double-Checked Locking* ou *Initialization-on-demand holder idiom*).
- Implementação do Factory Method garantindo que a classe cliente invoque apenas os contratos (interfaces), jamais as classes concretas de pagamento.

3. Classe de Simulação (Client / Main):

- Um roteiro de execução que simule a chegada de múltiplas transações financeiras distintas, comprove o uso dinâmico da fábrica e demonstre, através da impressão do *hashcode* ou identificador de memória, que todas as transações passaram pela exata mesma instância do componente de Compliance.